Presentation Agenda

PART 1 — FOUNDATIONS

01 Introduction to Test Case Development

02 Objectives of Test Case Development

03 Understanding Requirements & RTM

04 Structure of a Test Case — All 11 Fields

05 Types of Test Cases with Examples

06 Test Case Design Techniques

07 Best Practices for Writing Test Cases

08 Test Data Preparation

PART 2 — TOOLS & PROCESS

09 Manual vs Automated Test Cases

10 Selenium & Playwright Code Examples

11 Test Case Review & Optimization

12 Entry & Exit Criteria

13 Deliverables of Test Case Development

14 Sample Templates & Real-World Examples

15 Common Mistakes to Avoid

16 Summary & Key Takeaways

Introduction to Test Case Development — Definition

DEFINITION

A Test Case is a structured document that specifies the conditions, input data, execution steps, and expected results used to verify that a specific function or feature of a software application behaves as intended.

What to Test

The specific feature, function, or behaviour that needs to be verified — directly linked to a business or technical requirement.

How to Test

The exact steps a tester must follow to execute the test: navigating the UI, entering data, triggering actions, and observing outcomes.

Expected Outcome

The precise result the system must produce when the steps are executed correctly, forming the pass/fail benchmark.

Importance of Test Cases & Role in the STLC

WHY ARE TEST CASES IMPORTANT?

- Provide a repeatable, structured approach to testing — no ad-hoc guesswork
- Ensure every requirement is covered before software is released
- Serve as the primary communication bridge between developers and QA teams
- Enable consistent results regardless of who executes the test
- Create a paper trail for audit, compliance, and regression testing
- Help detect defects early — reducing the cost of fixing bugs post-release
- Support automation: well-written manual test cases translate directly into automated scripts

WHERE DOES IT FIT IN THE STLC?

1. Requirement Analysis
2. Test Planning
3. **Test Case Development** ← *YOU ARE HERE*
4. Test Environment Setup
5. Test Execution
6. Test Closure

Test Scenario vs Test Case vs Test Script — Key Differences

Aspect	Test Scenario	Test Case	Test Script
Definition	High-level description of WHAT to test	Detailed specification of HOW to test	Automated code that executes the test case
Detail Level	Low — broad and general	Medium/High — precise steps & data	Very High — code with assertions
Performed by	Business Analyst / QA Lead	QA Engineer / Test Analyst	Automation Engineer
Format	Plain text / bullet points	Structured table document	Code file (Java, JS, Python)
Tool	Confluence, Word, Excel	Excel, Jira, TestRail, Zephyr	Selenium, Playwright, JUnit
Example	'Test the login functionality of the application'	TC_001: Enter valid email + password → Click Login → Verify dashboard loads	driver.findElement(By.id('loginBtn')).click(); assertEquals(driver.getTitle(), 'Dashboard');

02

SECTION TWO

Objectives of Test Case Development

Why do we write test cases? What are we trying to achieve?

01 Requirement Coverage
Guarantee every business and functional requirement has at least one associated test case.

02 Functionality Validation
Confirm that each feature behaves exactly as specified in the requirements documentation.

03 Early Defect Detection
Identify bugs as early as possible — the earlier a defect is found, the cheaper it is to fix.

04 Software Quality Assurance
Provide objective evidence that the system meets defined quality standards before release.

05 Regression Assurance
Enable re-testing after code changes to ensure existing functionality has not been broken.

06 Audit & Compliance
Produce documentation trails required for regulated industries such as finance, health, and government.

Understanding Requirements Before Writing Test Cases

Business Requirements Document — BRD

Defines WHAT the business needs. Produced by Business Analysts in collaboration with stakeholders. Contains high-level goals, scope, and constraints. Written in business language — no technical implementation details.

Key Contents:

- Business goals & objectives
- In-scope and out-of-scope items
- Key stakeholder sign-offs
- Use cases & user stories
- Constraints & assumptions

Software Requirements Specification — SRS

Translates the BRD into technical specifications. Defines HOW the system must behave. Produced by Systems Analysts or Product Owners. Contains detailed functional and non-functional requirements.

Key Contents:

- System functional requirements
- Data input/output specifications
- Interface requirements
- Performance benchmarks
- Security and compliance requirements

Functional vs Non-Functional Requirements — What Can Be Tested?

Attribute	Functional Requirement	Non-Functional Requirement
Definition	Describes WHAT the system does	Describes HOW WELL the system does it
Focus	Features, functions, and behaviour	Performance, security, usability, reliability
Examples	User can log in with valid credentials Form validates required fields System sends confirmation email	Page loads in under 2 seconds 10,000 concurrent users supported Data encrypted with AES-256
Test Types	Functional Testing, Integration Testing, UAT	Performance, Load, Stress, Security, Usability Testing
Tools	Selenium, Playwright, Postman, JUnit	JMeter, Gatling, OWASP ZAP, Lighthouse
Testable?	Yes — clear pass/fail criteria	Yes — but requires measurable thresholds

QE TIP: A requirement is testable if it is specific, measurable, unambiguous, and has a clear expected outcome. Vague requirements must be clarified before test cases are written.

Requirement Traceability Matrix (RTM) — Definition & Purpose

RTM The Requirement Traceability Matrix (RTM) is a document that maps and traces each business/functional requirement to its corresponding test case(s). It ensures complete test coverage — that no requirement has been left untested.

Ensure Coverage

Verifies that 100% of documented requirements have at least one test case assigned to them — preventing gaps in testing.

Track Test Progress

Provides real-time visibility into which requirements have been tested, are in progress, or are blocked, giving QA leads status at a glance.

Impact Analysis

When a requirement changes, the RTM immediately shows which test cases are affected — enabling fast and accurate regression planning.

Audit & Compliance

Regulated industries (banking, healthcare, aviation) require traceability evidence to prove every requirement was tested before deployment.

RTM Columns: Req ID | Requirement Description | Test Case ID | Test Case Description | Status | Remarks

RTM — Real-World Example (E-Commerce Login Module)

Req ID	Requirement Description	Test Case ID	Test Case Description	Priority	Status
REQ-001	User must log in with valid email and password	TC_LOGIN_001	Verify login with valid credentials	High	Pass
REQ-001	User must log in with valid email and password	TC_LOGIN_002	Verify login with invalid password	High	Pass
REQ-001	User must log in with valid email and password	TC_LOGIN_003	Verify login with empty email field	Medium	Pass
REQ-002	System locks account after 3 failed login attempts	TC_LOGIN_004	Verify account lockout after 3 failed attempts	High	Pass
REQ-002	System locks account after 3 failed login attempts	TC_LOGIN_005	Verify lockout message is displayed	Medium	Pass
REQ-003	User must be able to reset password via email	TC_LOGIN_006	Verify password reset email is sent	High	In Progress
REQ-003	User must be able to reset password via email	TC_LOGIN_007	Verify reset link expires after 24 hours	Medium	Not Run
REQ-004	System must redirect to dashboard after successful login	TC_LOGIN_008	Verify dashboard redirect on login	High	Pass

Structure of a Test Case — Fields 1 to 6 (Detailed Explanation)

01 Test Case ID

DEF: A unique alphanumeric identifier assigned to each test case.

EG: Format: [MODULE]_[TYPE]_[NUMBER] — e.g., TC_LOGIN_001, TC_PAYMENT_NEG_003

WHY: Enables quick reference, traceability to RTM, and prevents duplicate test cases.

02 Test Scenario

DEF: A high-level description of the business function or feature being tested.

EG: Example: 'Verify login functionality for registered users'

WHY: Groups related test cases together; maps to requirements and user stories.

03 Test Description

DEF: A concise statement of exactly what this specific test case verifies.

EG: Example: 'Verify that a user can log in successfully with a valid email and correct password'

WHY: Distinguishes this test case from others in the same scenario; clarifies scope.

04 Preconditions

DEF: All conditions that must be true BEFORE the test case can be executed.

EG: Examples: Application is running, User account 'john@example.com' exists, User is on the Login page

WHY: Ensures the environment is correctly set up; prevents false failures due to environment issues.

05 Test Steps

DEF: Numbered, sequential, atomic actions the tester must perform to execute the test.

EG: 1. Navigate to /login 2. Enter 'john@example.com' 3. Enter 'SecurePass@123' 4. Click 'Login' button

WHY: Must be unambiguous — any tester (or automation tool) should produce identical results.

06 Test Data

DEF: The specific input values used during test execution.

EG: Email: john@example.com | Password: SecurePass@123 | URL: https://app.example.com/login

WHY: Separating test data from steps enables easy data-driven testing and reuse of the same steps with multiple data sets.

Structure of a Test Case — Fields 7 to 11 (Detailed Explanation)

07 Expected Result

DEF: The precise outcome that should occur if the software works correctly. Written BEFORE execution.

EG: 'User is redirected to the Dashboard page. Welcome message displays: Welcome, John!'

WHY: Defines the pass/fail benchmark — the tester compares actual vs expected to determine result.

08 Actual Result

DEF: What ACTUALLY happened when the test was executed. Filled in DURING or AFTER execution.

EG: Pass: 'User redirected to Dashboard as expected'
Fail: 'Error 500 displayed — server error'

WHY: Documents real system behaviour; drives defect reporting when it differs from expected result.

09 Status

DEF: The outcome classification after test execution.

EG: Pass | Fail | Blocked | Not Executed | In Progress | Deferred

WHY: Tracks overall testing progress; feeds into test summary reports and dashboards.

10 Priority

DEF: Indicates the ORDER in which test cases should be executed, based on business impact.

EG: High: Core business flow | Medium: Important feature | Low: Minor / cosmetic

WHY: Ensures the most critical functionality is tested first — critical in time-constrained sprints.

11 Severity

DEF: Indicates the IMPACT on the system if the tested feature fails.

EG: Critical: System crash/data loss | Major: Core function broken | Minor: Degraded UX | Trivial: Cosmetic

WHY: Guides the development team on defect fix priority; informs release go/no-go decisions.

Priority vs Severity — Key Distinction

A defect can be HIGH severity but LOW priority.
Example: A spelling error on the CEO page has HIGH priority (visibility) but LOW severity (no functionality broken).

Always consider both when filing a defect report.

Test Case — Complete Structured Example (Login Module)

Field	TC_LOGIN_001	TC_LOGIN_002	TC_LOGIN_003
Test Scenario	Login Functionality	Login Functionality	Login Functionality
Description	Login with valid credentials	Login with invalid password	Login with empty email field
Preconditions	App running; account john@example.com / SecurePass@123 exists; user on login page	App running; account exists; user on login page	App running; user on login page
Test Steps	1. Enter john@example.com 2. Enter SecurePass@123 3. Click Login	1. Enter john@example.com 2. Enter WrongPass999 3. Click Login	1. Leave Email field blank 2. Enter SecurePass@123 3. Click Login
Test Data	Email: john@example.com Pass: SecurePass@123	Email: john@example.com Pass: WrongPass999	Email: (empty) Pass: SecurePass@123
Expected Result	User redirected to Dashboard. Welcome, John! message shown.	Error displayed: 'Invalid email or password'. User stays on login page.	Validation error: 'Email is required'. Login button remains inactive.
Actual Result	User redirected to Dashboard as expected	Error message displayed as expected	Validation error shown as expected
Status	**Pass**	**Pass**	**Pass**
Priority	High	High	Medium
Severity	Critical	Major	Minor

05 SECTION FIVE

Types of Test Cases

Every testing scenario requires a different type of test case. Understanding them ensures complete coverage.

01 **Positive**
Valid inputs, happy path — verifies system works correctly under normal conditions

02 **Negative**
Invalid inputs, error paths — verifies system handles incorrect data gracefully

03 **Edge Case**
Extreme valid inputs at the outer limits of acceptable ranges

04 **Boundary Value**
Values at, just below, and just above defined boundaries

05 **Functional**
Verifies specific business logic and features against requirements

06 **Non-Functional**
Verifies performance, security, load capacity, and usability

Types of Test Cases — Detailed Examples (Password Field: 8–20 characters)

POSITIVE

Input: 'MyPass@123' (10 chars)
Step: Enter in password field and submit
Expected: Accepted — no validation error shown

NEGATIVE

Input: 'ab' (2 chars)
Step: Enter in password field and submit
Expected: Error — 'Password must be at least 8 characters'

EDGE CASE

Input: 'Aa1!Aa1!' (exactly 8 chars — minimum valid)
Step: Enter and submit
Expected: Accepted — just meets the minimum threshold

BOUNDARY VALUE

Test: 7 chars → Reject | 8 chars → Accept | 9 chars → Accept | 20 chars → Accept | 21 chars → Reject
Verify each boundary point individually

FUNCTIONAL

Input: valid 12-char password
Step: Complete registration flow
Expected: Password stored hashed, user account created, confirmation email sent

NON-FUNCTIONAL

Type: Performance
Action: 1,000 concurrent users submit login forms simultaneously
Expected: All responses return in < 3 seconds under load

06

SECTION SIX

Test Case Design Techniques

Systematic methods for deriving comprehensive test cases from requirements

01	02	03	04	05
Equivalence Partitioning	**Boundary Value Analysis**	**Decision Table Testing**	**State Transition Testing**	**Use Case Testing**
Divide input domain into equivalent classes; test one value from each class	Test values at, just below, and just above boundary limits	Map combinations of conditions and their resulting actions	Model software states and the events that trigger transitions	Test based on user-system interactions defined in use cases

Design Technique 1 — Equivalence Partitioning (EP)

EP divides all possible inputs into groups (partitions/equivalence classes) where every value in the group is expected to behave identically. Instead of testing every single value, you test one representative value from each partition — dramatically reducing the number of test cases without reducing coverage.

Example: Age Field (Valid range: 18–65 years)

Partition 1 (Invalid — Too Low)	Partition 2 (Invalid — Underage)	Partition 3 (VALID)	Partition 4 (Invalid — Too Old)	Partition 5 (Invalid — Out of Range)
Range:	**Range:**	**Range:**	**Range:**	**Range:**
< 0	**0 to 17**	**18 to 65**	**66 to 120**	**> 120**
Test values:	**Test values:**	**Test values:**	**Test values:**	**Test values:**
−5, −1	0, 10, 17	18, 40, 65	66, 90, 120	121, 999
Reject	**Reject**	**Accept ✓**	**Reject**	**Reject**

KEY INSIGHT: Testing one value per partition is equivalent to testing all values in that partition. 5 partitions = 5 test cases (not hundreds).

Design Technique 2 — Boundary Value Analysis (BVA)

BVA focuses on the edges (boundaries) of input ranges because defects are most commonly found AT boundaries, not in the middle of valid ranges. For every boundary, test the value just below, at, and just above the boundary point.

Example: Password field — minimum 8 characters, maximum 20 characters

Length	Test Value	Boundary Point	Result	Explanation
7 chars	'Aa1!Aa1'	*Just BELOW minimum*	REJECT ✗	One character short of the minimum — must be rejected
8 chars	'Aa1!Aa1!'	*AT minimum boundary*	ACCEPT ✓	Exactly the minimum — must be accepted
9 chars	'Aa1!Aa1!B'	*Just ABOVE minimum*	ACCEPT ✓	One above minimum — safely within range
19 chars	'Aa1!Aa1!BcDeFgHiJ'	*Just BELOW maximum*	ACCEPT ✓	One below maximum — safely within range
20 chars	'Aa1!Aa1!BcDeFgHiJk'	*AT maximum boundary*	ACCEPT ✓	Exactly the maximum — must be accepted
21 chars	'Aa1!Aa1!BcDeFgHiJkL'	*Just ABOVE maximum*	REJECT ✗	One character over maximum — must be rejected

RULE: BVA always produces 6 test cases per boundary pair (2 boundaries × 3 values each: min−1, min, min+1, max−1, max, max+1).

Design Technique 3 — Decision Table Testing

Decision Table Testing is used when the system behaviour depends on multiple conditions simultaneously. Each column in the table represents a unique combination of condition values (Y/N), and the resulting action for that combination.

Example: E-Commerce Checkout — Discount eligibility (Member + Promo Code + Cart Total > R500)

Condition / Rule	R1	R2	R3	R4	R5	R6	R7
Is Member?	Y	Y	Y	Y	N	N	N
Has Promo Code?	Y	Y	N	N	Y	N	N
Cart > R500?	Y	N	Y	N	Y	Y	N
RESULT	**20% OFF**	**15% OFF**	**10% OFF**	**5% OFF**	**5% OFF**	**No Deal**	**No Deal**

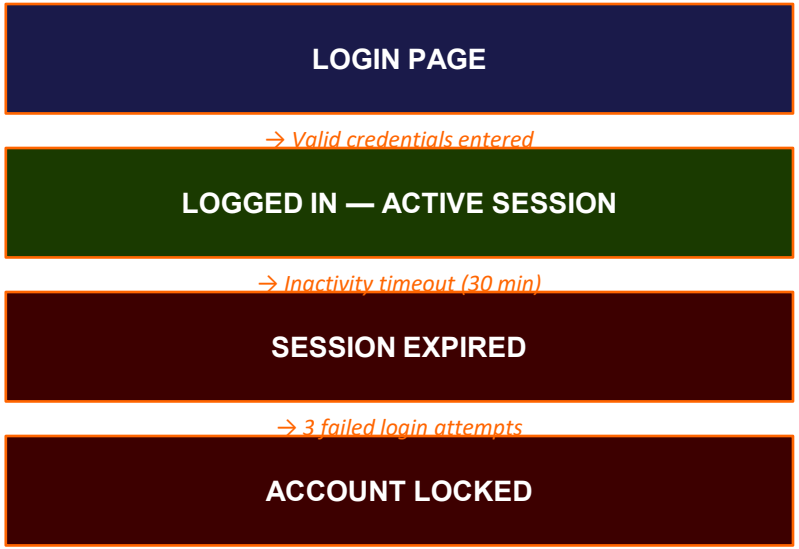
Each column = 1 unique test case. 7 columns = 7 test cases covering ALL condition combinations — no guessing.

FORMULA: 2^n combinations where n = number of conditions. 3 conditions = $2^3 = 8$ combinations (7 unique here as one combination is impossible).

Design Techniques 4 & 5 — State Transition & Use Case Testing

STATE TRANSITION TESTING

Tests how the system moves between defined states based on events/inputs.
Each state change = a test case.



USE CASE TESTING

Based on use case documents. Tests end-to-end user-system interactions.
Covers: Actor → Action → System Response → Outcome.

Use Case	UC-005: Place an Order
Actor	Registered Customer
Preconditions	User logged in; items in cart; payment method saved
Basic Flow	Select items → View cart → Apply coupon → Checkout → Confirm order → Receive confirmation email
Alt Flow 1	Coupon invalid → Show error → User continues without coupon
Alt Flow 2	Payment fails → Retry prompt → Order held pending payment
Postcondition	Order created in system; inventory decremented; email sent

07 SECTION SEVEN

Best Practices for Writing Effective Test Cases

1 Use Action Verbs

Begin every step with a clear verb: Navigate, Enter, Click, Verify, Assert. Never use passive or vague language.

2 One Expected Result Per TC

Each test case must test one specific thing. Multiple expected results per TC creates ambiguity in pass/fail analysis.

3 Make TCs Independent

No test case should depend on another TC having run first. Each TC must be executable in isolation.

4 Meaningful IDs

Use structured IDs: TC_[MODULE]_[TYPE]_[NUMBER]. Example: TC_LOGIN_NEG_003 instantly tells you it's a negative login test.

5 Cover Positive & Negative

For every positive scenario (happy path), always write at least one negative scenario (error path).

6 Document Preconditions

List ALL setup requirements: test account details, browser version, feature flags, environment URLs.

7 Trace to Requirements

Every test case must reference the requirement ID it verifies. If no requirement exists, question why you're testing it.

8 Peer Review

All test cases must be reviewed by a senior QA before execution. A second set of eyes catches ambiguity and missing cases.

Test Data Preparation — Types, Management & Importance

Test data is the specific set of inputs used to execute test cases. Poor test data leads to missed defects — even well-written test cases will fail to detect bugs if executed with the wrong data. Data must be prepared BEFORE test execution begins.

VALID DATA

- Correct format, within acceptable range
- Meets all field validation rules
- Example: Email = 'user@domain.com'
- Example: Age = 25
- Example: Phone = '0821234567' (10 digits)
- Used in positive test cases — system MUST accept

INVALID DATA

- Wrong format, out of range, or forbidden
- Example: Email = 'not-an-email'
- Example: Age = -5 or 999
- Example: Phone = 'ABCDEFGHIJ'
- SQL injection: admin'--
- Used in negative test cases — system MUST reject

EDGE / BOUNDARY DATA

- Values at or just outside boundary limits
- Example: Min password = 8 chars → test 7, 8, 9
- Example: Max file upload = 5MB → test 4.9MB, 5MB, 5.1MB
- Empty string, null, whitespace-only inputs
- Special characters: !@#$%^&*()
- Unicode & international characters: äöüñ 漢字

DATA MANAGEMENT: Never use real production data in test environments. Use anonymised/synthetic data. Store test data in shared repositories (Excel, TestRail, databases).

Manual vs Automated Test Cases — When to Use Each

Comparison Aspect	Manual Testing	Automated Testing
Definition	A human tester follows test case steps manually, observing and recording outcomes	A script (Selenium, Playwright) executes the test case automatically with no human intervention
Best For	Exploratory, usability, ad-hoc, one-time tests	Regression, smoke, load, repetitive, and data-driven tests
Speed	Slow — human execution time limits throughput	Very fast — can run 1,000s of tests in minutes
Accuracy	Prone to human error and fatigue over repeated execution	100% consistent — same inputs always produce same execution
Cost	Low initial cost; expensive at scale	High initial investment (scripting); low cost at scale
When to Automate?	—	When a test case will be run > 3 times, covers critical business flows, or is part of a regression suite
Tools	Excel/Sheets, Jira, TestRail, Zephyr	Selenium (Java/Python), Playwright (JS/TypeScript), JUnit, TestNG
Skills Required	Domain knowledge, analytical thinking	Programming, framework knowledge (Java, JS, Python)
Example	Manually verifying a new UI layout for brand consistency	Automated login regression test run after every deployment

SELENIUM | Java + TestNG | IntelliJ / Eclipse IDE

Automating TC_LOGIN_001 — Valid Login Test

```java
// TC_LOGIN_001 — Selenium WebDriver Test (Java + TestNG)
// Requirement: REQ-001 | Priority: High | Severity: Critical

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.*;

public class LoginTest {
    WebDriver driver;

    @BeforeMethod    // Runs BEFORE each test — sets up browser
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://app.example.com/login");
    }

    @Test(description = "TC_LOGIN_001: Login with valid credentials")
    public void testValidLogin() {
        driver.findElement(By.id("email")).sendKeys("john@example.com");
        driver.findElement(By.id("password")).sendKeys("SecurePass@123");
        driver.findElement(By.id("loginBtn")).click();
        Assert.assertEquals(driver.getTitle(), "Dashboard", "Login failed!");
    }

    @AfterMethod     // Runs AFTER each test — closes browser
    public void tearDown() { driver.quit(); }
```

PLAYWRIGHT | JavaScript / TypeScript | VS Code / IntelliJ

TC_LOGIN_001 + TC_LOGIN_002 — Valid & Invalid Login

```
// Playwright Test — TC_LOGIN_001 & TC_LOGIN_002
// Framework: Playwright + TypeScript | IDE: VS Code

import { test, expect } from '@playwright/test';

const BASE_URL = 'https://app.example.com/login';

test.describe('Login Module — TC_LOGIN_001 & TC_LOGIN_002', () => {

  // TC_LOGIN_001: Positive — Valid Credentials
  test('should login successfully with valid credentials', async ({ page }) => {
    await page.goto(BASE_URL);
    await page.fill('#email', 'john@example.com');
    await page.fill('#password', 'SecurePass@123');
    await page.click('#loginBtn');
    await expect(page).toHaveTitle('Dashboard');
    await expect(page.locator('.welcome')).toContainText('Welcome, John!');
  });

  // TC_LOGIN_002: Negative — Invalid Password
  test('should show error for invalid password', async ({ page }) => {
    await page.goto(BASE_URL);
    await page.fill('#email', 'john@example.com');
    await page.fill('#password', 'WrongPass999');
    await page.click('#loginBtn');
    await expect(page.locator('.error-msg')).toBeVisible();
    await expect(page.locator('.error-msg')).toHaveText('Invalid email or password');
  });
});
```

Test Case Review & Optimization — Quality Control Process

Test case documents are NOT final upon first draft. A structured review process must be followed before any test case is approved for execution.

01 Self-Review (Author)

- Re-read every step — would a new tester understand this?
- Verify all 11 fields are completed
- Check test data is realistic and varied
- Confirm expected result is specific, not vague
- Ensure TC is traceable to a requirement

02 Peer Review (Fellow QA)

- Independent QA reads and attempts to execute mentally
- Identifies ambiguous steps or missing preconditions
- Flags duplicate test cases
- Suggests missing edge cases and negative scenarios
- Raises questions on unclear expected results

03 QA Lead Validation

- Reviews coverage against the RTM
- Checks priority and severity assignments
- Confirms test data is appropriate
- Approves or requests changes
- Signs off test cases as ready for execution

04 Optimization (Ongoing)

- Remove redundant or duplicate TCs
- Merge similar TCs where appropriate
- Update TCs when requirements change
- Archive obsolete test cases
- Tag TCs for automation candidates

Entry & Exit Criteria for Test Case Development

Entry criteria define the conditions that must be met BEFORE test case development can begin. Exit criteria define when the test case development phase is considered COMPLETE.

ENTRY CRITERIA — Must Be True BEFORE Starting

- Business Requirements Document (BRD) is approved and baselined
- Software Requirements Specification (SRS) has been reviewed and signed off
- Test Plan has been created and approved by QA Lead and Project Manager
- Requirement Traceability Matrix (RTM) has been created (initial draft)
- Test environment details and access credentials are available
- Test data sources and access permissions have been confirmed
- Tools configured: Jira, TestRail/Zephyr, Selenium/Playwright frameworks

EXIT CRITERIA — Must Be True BEFORE Proceeding

- 100% of testable requirements have at least one mapped test case
- All 11 test case fields are fully and accurately completed
- All test cases have passed peer review and QA Lead sign-off
- RTM has been updated to reflect all test case IDs and their status
- Test data sets have been prepared, validated, and stored in repository
- Test cases have been uploaded to the test management tool (Jira/TestRail)
- Test case coverage report has been generated and shared with stakeholders

Deliverables of the Test Case Development Phase

D1

Test Case Document

The primary deliverable. A structured document (Excel, TestRail, Zephyr) containing all test cases with all 11 fields completed for every testable requirement.

Format: Excel (.xlsx), TestRail, Zephyr Scale, or Confluence table
Naming: TC_[ProjectName]_[Module]_v[1.0].xlsx

D2

Requirement Traceability Matrix (RTM)

An updated RTM showing the mapping between every requirement ID and its corresponding test case IDs, priorities, and execution status.

Format: Excel or Confluence table
Columns: Req ID | Req Description | TC ID(s) | TC Status | Remarks

D3

Test Data Repository

A curated collection of test data sets (valid, invalid, edge) aligned to each test case. Must include data for positive, negative, and boundary scenarios.

Format: Excel sheets, CSV files, or database scripts
Location: Shared team repository (Confluence, SharePoint)

D4

Test Case Review Sign-Off Sheet

A document confirming that all test cases have been peer-reviewed, QA Lead validated, and approved. Includes reviewer name, date, and approval status.

Format: Physical or digital sign-off form
Required: Author, Peer Reviewer, QA Lead signatures

Sample Test Case Template — Blank (Use This as Your Standard Format)

Test Case ID	Test Scenario	Preconditions	Test Steps	Test Data	Expected Result	Status	Prio
TC_[MOD]_001	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[ent valu
TC_[MOD]_002	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[ent valu
TC_[MOD]_003	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[ent valu
TC_[MOD]_004	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[ent valu
TC_[MOD]_005	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[enter value]	[ent valu

Field Descriptions:

Test Case ID: Unique identifier (TC_MODULE_001) | Test Scenario: High-level feature area | Preconditions: Setup required before execution | Test Steps: Numbered sequential actions | Test Data: Specific input values | Expected Result: What SHOULD happen | Status: Pass/Fail/Blocked/Not Run | Priority: High/Med/Low | Severity: Critical/Major/Minor/Trivial

Sample Test Cases — Login Functionality (Fully Completed Real-World Example)

TC ID	Scenario	Preconditions	Test Steps	Test Data	Expected Result	Status
TC_LOGIN_001	Valid Login	App running; account exists	1. Go to /login 2. Enter email 3. Enter password 4. Click Login	john@example.com SecurePass@123	Redirected to Dashboard. 'Welcome, John!' shown.	Pass
TC_LOGIN_002	Invalid Password	App running; account exists	1. Go to /login 2. Enter email 3. Enter wrong password 4. Click Login	john@example.com WrongPass999	Error: 'Invalid email or password'. User stays on login page.	Pass
TC_LOGIN_003	Empty Email	App running; user on login page	1. Go to /login 2. Leave email blank 3. Enter password 4. Click Login	Email: (blank) SecurePass@123	Validation error: 'Email is required'. Login disabled.	Pass
TC_LOGIN_004	Account Lockout	Account exists; 0 failed attempts	1. Enter wrong password 2. Repeat 2 more times 3. Attempt 4th login	Incorrect password used 3 times	Account locked after 3rd attempt. Message: 'Account locked. Contact support.'	Pass
TC_LOGIN_005	SQL Injection	App running; user on login page	1. Enter SQL payload in email 2. Enter any password 3. Click Login	Email: admin'-- Pass: anything	Login rejected. No DB error exposed. Generic error shown.	Pass
TC_LOGIN_006	Password Reset	Account exists; user on login page	1. Click 'Forgot Password' 2. Enter registered email 3. Click 'Send Reset Link'	john@example.com	Success message: 'Reset link sent to john@example.com'. Email received within 60s.	Fail

Sample Defect / Bug Report — Real-World Example

When a test case FAILS, a defect report must be immediately logged in the defect tracking system (Jira). A complete, accurate bug report is critical for fast resolution.

Field	Details
Bug ID	BUG-00147
Title / Summary	Password Reset email not delivered — TC_LOGIN_006 FAILED
Module	Authentication — Password Reset
Environment	QA Environment | Chrome 124 | Windows 11 | App v2.3.1-RC1
Reported By	Jane Doe (QA Engineer) — 2025-08-14 09:32
Assigned To	Mark Smith (Backend Developer)
Steps to Reproduce	1. Navigate to https://qa.app.example.com/login 2. Click 'Forgot Password' 3. Enter john@example.com in the email field 4. Click 'Send Reset Link' 5. Wait 5 minutes and check email inbox
Expected Result	User receives password reset email within 60 seconds containing a valid reset link
Actual Result	**No email received after 5 minutes. No error message shown on screen. UI shows 'Reset link sent' success message incorrectly.**
Severity	**CRITICAL — Core authentication feature broken; users cannot recover accounts**
Priority	**HIGH — Must be resolved before next sprint release (Release v2.3.1)**
Status	OPEN — Awaiting developer investigation

Real-World Workflow — From Requirements to Defect Logging

1 Receive Requirements

QA team receives BRD + SRS documents. QA Lead schedules a requirements walkthrough meeting with BA and developers to clarify ambiguities and identify testable requirements.

2 Create RTM

QA Analyst creates the initial RTM — listing every requirement ID and description. Test Case ID columns are left blank and populated as test cases are written.

3 Write Test Cases

QA Engineer writes detailed test cases using the standard template. Covers positive, negative, edge, and boundary scenarios for all testable requirements. Uses BVA, EP, and Decision Tables.

4 Review & Approval

Peer review → QA Lead sign-off → Update RTM with TC IDs and coverage percentage. Any issues flagged during review are fixed before approval.

5 Execute Test Cases

QA team executes test cases in the test environment. Actual results are recorded for each TC. Automation scripts (Selenium/Playwright) run as part of the CI/CD pipeline.

6 Log Defects

Any TC that FAILS immediately triggers a defect report in Jira. Report includes: Bug ID, reproduction steps, severity, priority, screenshots, and logs. Developer is assigned and notified.

7 Retest & Close

After developer fixes the defect, QA retests using the same TC that originally failed. If retest passes, bug is closed and TC status updated to Pass. RTM is updated accordingly.

16

SECTION SIXTEEN

Common Mistakes in Test Case Development

Avoid these pitfalls — they are the most common causes of defect escapes and release failures

✗ Missing Edge Cases

Only testing the 'happy path'. Always ask: what happens at extremes? What if the field is empty, max length, or contains special characters?

✗ Vague Test Steps

Steps like 'verify login works' are untestable. Use specific actions: Navigate to /login, Enter 'john@example.com', Click '#loginBtn', Assert page title = 'Dashboard'.

✗ No Preconditions

Assuming the environment will always be in the right state. Missing preconditions cause false failures and wasted time debugging test setup.

✗ No Requirement Traceability

Test cases that cannot be mapped to a requirement are wasteful. Every TC must reference at least one Req ID — without traceability you cannot prove coverage.

✗ Redundant Test Cases

Duplicate TCs bloat the test suite and slow execution. Regularly review and merge overlapping cases. Each TC must add unique value.

✗ Ignoring Negative Scenarios

Only writing positive (happy path) TCs. Defects hide in error handling, invalid input processing, and boundary breaches — all negative scenarios.

✗ Outdated Test Cases

Failing to update TCs when requirements change. Stale test cases produce misleading Pass results. Assign a TC owner responsible for maintenance.

✗ Poor Test Data Selection

Using only one or two test values. Select data that covers valid, invalid, edge, and boundary cases. Use realistic data that mirrors production patterns.

SUMMARY

Key Takeaways

- Test cases are the backbone of structured quality assurance — without them, testing is guesswork
- Every test case must be traceable to a business or functional requirement via the RTM
- A complete test case has 11 fields: ID, Scenario, Description, Preconditions, Steps, Data, Expected, Actual, Status, Priority, Severity
- Use design techniques (EP, BVA, Decision Tables, State Transition, Use Cases) to maximize coverage with minimum test cases
- Always write positive AND negative test cases — defects hide in error handling and boundary conditions
- Well-written manual test cases translate directly into automated scripts (Selenium/Playwright)
- Test cases must be reviewed, approved, and maintained — they are living documents that evolve with the software
- A failed test case = a defect report in Jira — complete, accurate bug reports drive fast resolution
- Coverage quality matters more than quantity — fewer precise test cases > many vague ones